"...one of the most highly regarded and expertly designed C++ library projects in the world."
— Herb Sutter and Andrei Alexandrescu, C++ Coding Standards

Quick Start

This Quick Start section shows a simple way to create a typical R-tree and perform spatial query.

The code below assumes that following files are included and namespaces used.

```
#include <boost/geometry.hpp>
#include <boost/geometry/geometries/point.hpp>
#include <boost/geometry/geometries/box.hpp>

#include <boost/geometry/index/rtree.hpp>

// to store queries results
#include <vector>

// just for output
#include <iostream>
#include <boost/foreach.hpp>

namespace bg = boost::geometry;
namespace bgi = boost::geometry::index;
```

Typically you'll store e.g. `std::pair<Box, MyGeometryId>` in the R-tree. `MyGeometryId` will be some identifier of a complex `Geometry` stored in other container, e.g. index type of a `Polygon` stored in the vector or an iterator of list of `Rings`. To keep it simple to define `Value` we will use predefined Box and unsigned int.

```
typedef bg::model::point<float, 2, bg::cs::cartesian> point;
typedef bg::model::box<point> box;
typedef std::pair<box, unsigned> value;
```

R-tree may be created using various algorithm and parameters. You should choose the algorithm you'll find the best for your purpose. In this example we will use quadratic algorithm. Parameters are passed as template parameters. Maximum number of elements in nodes is set to 16.

```
// create the rtree using default constructor
bgi::rtree< value, bgi::quadratic<16> > rtree;
```

Typically `Values` will be generated in a loop from e.g. `Polygons` stored in some other container. In this case `Box` objects will probably be created with `geometry::envelope()` function. But to keep it simple let's just generate some boxes manually and insert them into the R-tree by using `insert()` method.

```
// create some values
for ( unsigned i = 0 ; i < 10 ; ++i )
{
    // create a box
    box b(point(i + 0.0f, i + 0.0f), point(i + 0.5f, i + 0.5f));
    // insert new value
    rtree.insert(std::make_pair(b, i));
}
```

There are various types of spatial queries that may be performed, they can be even combined together in one call. For simplicity, we use the default one. The following query return values intersecting a box. The sequence of `Values` in the result is not specified.

```
// find values intersecting some area defined by a box
box query_box(point(0, 0), point(5, 5));
std::vector<value> result_s;
rtree.query(bgi::intersects(query_box), std::back_inserter(result_s));
```

Other type of query is k-nearest neighbor search. It returns some number of values nearest to some point in space. The default knn query may be performed as follows. The sequence of `Values` in the result is not specified.

```
// find 5 nearest values to a point
std::vector<value> result_n;
rtree.query(bgi::nearest(point(0, 0), 5), std::back_inserter(result_n));
```

At the end we'll print results.

```
// display results
std::cout << "spatial query box:" << std::endl;
std::cout << bg::wkt<box>(query_box) << std::endl;
std::cout << "spatial query result:" << std::endl;
BOOST_FOREACH(value const& v, result_s)
    std::cout << bg::wkt<box>(v.first) << " - " << v.second << std::endl;

std::cout << "knn query point:" << std::endl;
std::cout << bg::wkt<point>(point(0, 0)) << std::endl;
std::cout << "knn query result:" << std::endl;
BOOST_FOREACH(value const& v, result_n)
    std::cout << bg::wkt<box>(v.first) << " - " << v.second << std::endl;
```

More

More information about the R-tree implementation, other algorithms and queries may be found in other parts of this documentation.